

Flappy Bird Game Using MASM

Group Members

[(Raed Ovais-24K-1033)]

(Rahim Hasan-24K-0975)

(Saad Tahir-24K-0851)

1. Introduction

- **Background:** This project focuses on the use of CPU interrupts and dos commands cleverly to make any imaginable program.
- **Problem Statement:** The challenge lies in creating a modern, interactive game (Flappy Bird) on a very old and low-level platform—x86 architecture using Assembly. We will need to manually handle graphic rendering, sound effects, and user input. This introduces significant complexity as Assembly requires precise control over each instruction executed on the CPU.
- **Objectives:** The goal of this project is to build a flappy bird game using Assembly interrupts in MASM. The game will have colour as well as audio, all of which can be executed via interrupts and dos commands.
- **Additional Features:**
 1. run-time theme change
 2. An option to change the character
 3. Storage of high-score using a file
 4. Commentary when certain movements/scores are reached

2. Scope of the Project

- **Inclusions:** Visual Studio, Irvine Library, MASM, use of interrupts
- **Exclusions:** The project will be limited to x86 architecture.

3. Project Description

- **Overview:** The project involves building a simplified version of the Flappy Bird game using Assembly language. The game will be played via keyboard input, with a bird navigating through columns. The project will also include the creation of graphical sprites for the bird and obstacles, background music, and sound effects—all generated using CPU interrupts and DOS commands.
 - **Technical Requirements:** A PC running MS-DOS or an emulator for x86 architecture, as well as basic input/output capabilities (keyboard, display, sound output). Microsoft Visual Studio (for editing and compiling), MASM (Assembler), and Irvine Library
- Project Phases:**
1. Research: Understand how Assembly language interacts with hardware, study interrupts in MASM, and familiarize with the functionality of the Irvine

Library.

2. Planning: Define the core game mechanics (gameplay, graphics, controls), and establish project milestones.
3. Design: Layout the structure of the code, creating the game loop, input processing, and graphics handling.
4. Implementation: Write and test the Assembly code for each component, including the bird's movement, background music, and the screen rendering logic.
5. Testing & Debugging: Run the game in an emulator, fine-tuning performance and fixing bugs.

4. Methodology

- **Approach:** The project will use an iterative approach where tasks are divided into smaller chunks.
- **Team Responsibilities:**
 1. Raed: Research and design the game loop and controls, focusing on handling user input and basic game logic.
 2. Rahim: Develop the graphics rendering system, creating the sprites and using interrupts to control screen output.
 3. Saad: Implement the sound system using DOS commands, as well as integrating the game mechanics.

5. Expected Outcomes

- **Deliverables:**
 1. A working Flappy Bird game, executable in DOS or an x86 emulator.
 2. A short project report documenting the process, challenges, and solutions.
 3. User instructions for running and playing the game.
- **Relevance:** It offers an in-depth understanding of how software interacts with hardware components (e.g., graphics card, sound card) and challenges the team to develop problem-solving skills in a constrained environment.

6. Resources Needed

1. **Online tutorials or textbooks on Assembly language and x86 architecture.**
2. **Instructor guidance for debugging and troubleshooting low-level Assembly code.**
3. **Emulators for testing the game on virtual x86 systems.**